

LAB #2—Timer & Interrupts

Prof. Kizito NKURIKIYEYEU

March 26, 2025

1 LAB OBJECTIVES

- Develop a comprehensive understanding of timer peripherals in microcontrollers and their applications
- Analyze the limitations and inefficiencies of polling-based input detection
- Implement and evaluate interrupt-driven I/O for responsive embedded systems
- Design robust embedded software that handles asynchronous events efficiently
- Quantify and optimize CPU utilization and power consumption in embedded applications
- Implement proper solutions for common embedded systems challenges including race conditions, critical sections, and shared resource management
- Apply formal state machine modeling to embedded software design

2 PREREQUISITE KNOWLEDGE

Before attempting this lab, ensure you understand:

- Basic digital I/O operations on the ATmega328 microcontroller
- C programming fundamentals for embedded systems
- Binary and hexadecimal number systems for register manipulation
- Basic concepts of microcontroller architecture (CPU, memory, peripherals)

3 BACKGROUND

Embedded systems frequently need to monitor inputs (buttons, sensors) while simultaneously maintaining other operations (updating displays, controlling actuators). Two fundamental approaches exist for handling such requirements:

- **Polling:** Repeatedly checking input states in a loop
- **Interrupt-driven:** Configuring the hardware to notify the CPU when events occur

Each approach has distinct implications for system performance, power consumption, responsiveness, and software complexity. This lab will provide you with a practical understanding of these implications through implementation and analysis.

4 HARDWARE SETUP

Four LEDs and two push-button switches are connected to an ATmega328 MCU as follows:

- LED 1 (D1, green) connected to PB0
- LED 2 (D2, red) connected to PB1
- LED 3 (D3, blue) connected to PB2
- LED 4 (D4, yellow) connected to PB3
- Push button switch (SW1) connected to PD2 (INT0)
- Push button switch (SW2) connected to PD3 (INT1)

Note: The pins for the push buttons have been changed from the original specification to utilize the external interrupt pins (INT0 and INT1) on the ATmega328.

5 EXERCISE 1: Polling implementation and analysis

5.1 Requirements

Implement a system with the following behavior using a polling-based approach:

- All LEDs are OFF at system initialization
- LED1 (green) and LED4 (yellow) blink continuously at precisely 1 Hz (500ms ON, 500ms OFF) in phase with each other
- While SW1 is pressed, LED2 (red) is ON; when released, LED2 is OFF
- When SW1 is pressed and then released, LED3 (blue) turns ON for exactly one second, then turns OFF
- The system must operate indefinitely without failure

5.2 Required reading

- Understand the AVR port. Read [this tutorial](https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/8-bit-avr/structure/reports/)¹
- Read chap 3 (Digital Output) in the textbook Make: AVR Programming by Elliot Williams
- Understand bit twiddling. [Read this lecture note](#)
- Read chap 4 (Bit Twiddling) in the textbook Make: AVR Programming by Elliot Williams
- Read chap 6 (Digital Input) in the textbook Make: AVR Programming by Elliot Williams

5.3 Implementation Tasks

1. Design a formal state machine that represents your system's operation, using standard state diagram notation. Your state machine should include:
 - All possible states
 - Transition conditions between states
 - Actions performed in each state
 - Initial state clearly marked
2. Using your state machine as a guide, implement the system in C for the ATmega328 microcontroller.

¹ <https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/8-bit-avr/structure/reports/>

3. Instrument your code to track CPU utilization by toggling a designated pin (PC0) whenever the CPU is in an active processing state versus an idle waiting state. This will be used for analysis in Proteus.
4. Create a Proteus simulation of your implementation and verify correct operation.

5.4 Analysis Requirements

After implementing your polling-based solution, perform the following analysis:

1. Calculate the theoretical maximum polling loop frequency based on your code's execution path.
2. Using Proteus's virtual logic analyzer, measure the actual polling frequency achieved in your implementation (by analyzing the transitions on PC0).
3. Estimate what percentage of CPU time is spent on:
 - Actual computation/decision making
 - Idle waiting for time delays
 - Idle polling for input changes
4. Identify and describe at least three specific limitations of your polling approach. For each limitation:
 - Explain the root cause
 - Describe a scenario where this limitation would cause a failure
 - Propose a mitigation strategy
5. Calculate the minimum, maximum, and average response time to button presses in your implementation. Explain what causes the variation.
6. If this system were battery-powered, estimate relative power consumption based on CPU activity levels.

5.5 Advanced Analysis

1. Add a 10ms software debounce routine to your polling implementation. Compare the system's responsiveness before and after adding debounce.
2. Extend your system to handle a more complex sequence: if SW1 is pressed three times within 2 seconds, toggle LED4's blinking frequency between 1 Hz and 2 Hz. Document any challenges encountered when implementing this feature using polling.
3. Calculate the theoretical maximum number of additional tasks your system could handle without missing button presses, assuming each task takes 1ms to execute.

Notes:

- Use the built-in `_delay_ms()` function for timing in this exercise to establish a baseline implementation.
- Document any assumptions or approximations made in your calculations.
- Include commented code and clear explanations for how you instrumented CPU utilization measurement.

6 EXERCISE 2: Timer-based implementation

6.1 Background

In this exercise, you will address the limitations of polling by implementing a timer-based solution. The ATmega328 has several timers that can be configured to generate precise time intervals without blocking the CPU.

6.2 Required Reading

Before attempting this exercise, review:

- Read Dean Camera's Newbie's Guide to AVR Timers, which is [available here](#)²
- Read chap 9 (Introduction to the Timer/Counter Hardware) in the textbook Make: AVR Programming by Elliot Williams

6.3 Timer Configuration Requirements

Implement the same system behaviors as in Exercise 1, but with the following modifications:

1. Use Timer1 to generate a precise 1ms system tick interrupt. This will serve as the time base for all timing operations.
2. Using this timer-based approach, implement:
 - A non-blocking delay mechanism that allows multiple timed events to run concurrently
 - Precise 1 Hz blinking for LED1 and LED4
 - Exact 1-second ON duration for LED3 after SW1 release
3. Continue using polling for switch input detection in this phase to isolate the benefits of timer-based timing.
4. Use pin PC1 (PORT C, Pin 1) and an LED to signal whenever the timer interrupt service routine (ISR) is executing, allowing measurement of ISR overhead.

6.4 Implementation Tasks

1. Update your state machine from Exercise 1 to incorporate the timer-based approach.
2. Implement the updated design using Timer1 configured for 1ms interrupts.
3. Create a generalized non-blocking delay function that can be used for multiple independent delays.
4. Design and implement a simple scheduler that:
 - Maintains a list of timed tasks
 - Executes each task when its time arrives
 - Allows tasks to be added, removed, and modified dynamically
5. Using your scheduler, implement the LED blinking and timing behaviors specified above.
6. Demonstrate correct operation in Proteus simulation.

6.5 Analysis Requirements

After implementing the timer-based solution:

1. Using Proteus’s virtual logic analyzer, measure:
 - Timer ISR execution time
 - Timer ISR frequency stability (jitter)
 - Main loop execution frequency
2. Compare and contrast with your Exercise 1 implementation:
 - CPU utilization (percentage of time spent in active processing)
 - Timing accuracy for the 1 Hz blinking and 1-second delay
 - Maximum number of additional tasks the system could handle
3. Calculate the timing resolution and overhead of your scheduler.
4. Analyze and explain any tradeoffs or consequences of your timer-based implementation.

7 EXERCISE 3: Interrupt-driven input

7.1 Background

In this exercise, you will fully eliminate polling by using external interrupts to detect button presses. This approach allows the CPU to remain in a low-power state until an event occurs that requires attention.

7.2 Required Reading

Before attempting this exercise, review:

- Read Dean Camera’s Newbie’s Guide to AVR Interrupts, which is [available here](#)
- Read chap 8 (Hardware Interrupts) in the textbook Make: AVR Programming by Elliot Williams

7.3 System Requirements

Implement the same system behaviors as in previous exercises, but with these modifications:

1. Use external interrupts (INT0 and INT1) to detect button presses and releases.
2. Configure INT0 (connected to SW1) to trigger on both rising and falling edges.
3. Configure INT1 (connected to SW2) to trigger on falling edge only.
4. Extend system functionality:
 - When SW2 is pressed, toggle the blinking rate of LED1 and LED4 between 1 Hz and 2 Hz
 - If both SW1 and SW2 are pressed simultaneously, perform a pattern display across all LEDs for 3 seconds
 - Implement a “sleep mode” that activates after 10 seconds of inactivity, turning off all LEDs until a button is pressed
5. Implement proper software debouncing in the interrupt handlers.
6. Use pin PC2 to signal whenever an external interrupt service routine is executing.

7.4 Implementation Tasks

1. Update your state machine to incorporate external interrupt-driven input detection.

2. Implement the interrupt service routines (ISRs) for INT0 and INT1.
3. Design and implement a robust debouncing mechanism that works within the interrupt-driven paradigm.
4. Add support for detecting both simultaneous and sequential button presses.
5. Integrate sleep mode functionality using the ATmega328's power management features.
6. Combine your timer-based scheduler from Exercise 2 with the new interrupt-driven input system.
7. Demonstrate correct operation in Proteus simulation.

7.5 Analysis Requirements

After implementing the fully interrupt-driven solution:

1. Measure and document:
 - External interrupt latency (time from button press to ISR execution)
 - Debounce mechanism effectiveness
 - Response time for all system behaviors
2. Compare your three implementations (polling, timer-based, fully interrupt-driven) in terms of:
 - Code complexity (lines of code, cyclomatic complexity)
 - CPU utilization
 - Input responsiveness
 - Timing accuracy
 - Estimated power efficiency
3. Identify and analyze any race conditions in your system:
 - Between timer ISR and external interrupt ISRs
 - Between different external interrupt ISRs
 - Between ISRs and main loop processing
4. Document your techniques for preventing or mitigating these race conditions.

8 EXERCISE 4: Traffic light controller

8.1 Background

Traffic light controllers are critical embedded systems that manage traffic flow and pedestrian safety at intersections. This exercise will challenge you to design and implement a traffic light controller for a four-way intersection with pedestrian crossings, applying your knowledge of AVR I/O operations, timers, and interrupts.

8.2 Hardware Setup

Configure your Proteus simulation with the following components:

- **Traffic Lights (12 LEDs total):**
 - North direction: Red, Yellow, Green LEDs connected to PB0, PB1, PB2
 - East direction: Red, Yellow, Green LEDs connected to PB3, PB4, PB5
 - South direction: Red, Yellow, Green LEDs connected to PC0, PC1, PC2
 - West direction: Red, Yellow, Green LEDs connected to PC3, PC4, PC5

- **Pedestrian Signals (4 LEDs):**
 - North crossing: Red/Green pedestrian LEDs connected to PD0, PD1
 - East crossing: Red/Green pedestrian LEDs connected to PD2, PD3
- **Pedestrian Call Buttons (4 buttons):**
 - North crossing button connected to PD4 (with pull-up)
 - East crossing button connected to PD5 (with pull-up)
 - South crossing button connected to PD6 (with pull-up)
 - West crossing button connected to PD7 (with pull-up)
- **Pedestrian Crossing Buzzer:**
 - Connected to PA0 for audible pedestrian crossing indication
- **7-Segment Display:**
 - Connected to PA1-PA7 to display countdown timers

8.3 System Requirements

Implement a traffic light controller with the following features:

1. **Basic Traffic Light Sequence:**
 - Implement a standard traffic light sequence for a four-way intersection
 - North/South and East/West directions operate as synchronized pairs
 - Proper sequence: Green → Yellow → Red, with appropriate timing
 - Green phase: 20 seconds, Yellow phase: 5 seconds, All-Red phase: 2 seconds
 - System should run indefinitely maintaining proper traffic flow
2. **Pedestrian Crossing System:**
 - Pedestrians can request to cross by pressing the corresponding button
 - Pedestrian signals show "Don't Walk" (red) or "Walk" (green)
 - When a crossing is requested, the system should:
 - Complete the current traffic cycle
 - Transition to a pedestrian crossing phase for the requested direction
 - Display a countdown timer on the 7-segment display showing remaining crossing time
 - Activate the buzzer during the pedestrian crossing phase
 - Pedestrian crossing time: 10 seconds
 - Final 3 seconds should have a faster buzzer pattern to indicate "hurry up"
3. **System Modes:**
 - Normal Mode: Standard sequence with 20-second green phases
 - Rush Hour Mode: Extended green phases (30 seconds) for the main road (North/South)
 - Night Mode: Flashing yellow for main road, flashing red for side road
 - Implement a way to switch between modes (can be simulated by a specific button combination)
4. **Emergency Vehicle Override:**
 - Implement an emergency mode that can be triggered (simulated by a specific button combination)
 - When triggered, safely transition all lights to red except for the specified direction
 - After emergency mode ends, safely resume normal operation

8.4 Implementation Requirements

Your implementation must adhere to these technical requirements:

1. **Timer Usage:**
 - Use Timer1 to create a precise 1-second time base for the traffic light timings
 - All timing must be non-blocking and interrupt-driven
 - No use of delay functions (`_delay_ms()`) for timing
2. **State Machine Design:**
 - Implement the system as a well-defined state machine
 - Document all states and transitions in your design
 - Ensure safe transitions between all states (no unsafe traffic signal combinations)
 - Handle edge cases such as multiple pedestrian requests
3. **Efficient I/O Operations:**
 - Use direct port manipulation for all I/O operations
 - Implement efficient bit operations for controlling multiple LEDs
 - Create helper macros/functions for common operations
 - Ensure proper initialization of all I/O pins
4. **Pedestrian Button Interface:**
 - Implement proper debouncing for all pedestrian buttons
 - Use interrupts to detect button presses for responsive operation
 - Store and prioritize pedestrian crossing requests
 - Provide visual feedback when a button press is registered

8.5 7-Segment Display Requirements

1. **Countdown Timer:**
 - Display the remaining time for the current green or pedestrian crossing phase
 - Create an efficient 7-segment decoder function
 - Ensure the display is clear and updates every second
 - Implement a flashing display for the final 3 seconds of pedestrian crossing
2. **System Status Indication:**
 - Use the display to indicate the current system mode
 - Show special patterns for emergency mode
 - Implement error codes for any system fault conditions

8.6 Buzzer Control Requirements

1. **Pedestrian Crossing Indication:**
 - Generate regular beeps during the pedestrian crossing phase
 - Increase the beep frequency during the final 3 seconds
 - Use timer-based tone generation (no delay functions)
 - Ensure the buzzer is only active during actual crossing phases
2. **System Alerts:**
 - Use distinct sound patterns for mode changes
 - Generate a special alert sound for emergency vehicle override
 - Ensure all sound patterns are non-blocking

8.7 Safety Requirements

1. Fail-Safe Operation:

- Ensure that conflicting green signals are never active simultaneously
- Implement an all-red safety phase between directional changes
- Verify that pedestrian walk signals never conflict with vehicle green signals
- Add a watchdog timer to reset the system in case of software failure

2. Error Handling:

- Define error conditions and appropriate system responses
- Implement system diagnostics to detect faulty operation
- Create a safe mode that the system can fall back to if errors are detected

8.8 Analysis and Testing Requirements

For your implementation:

1. State Verification:

- Create a test plan that verifies all possible state transitions
- Verify that all timing requirements are met accurately
- Test pedestrian request handling under various traffic conditions
- Verify proper operation of all special modes

2. Safety Testing:

- Test that conflicting signals are never activated
- Verify proper all-red safety phases
- Test system recovery from simulated errors
- Verify that pedestrian signals operate safely

3. Documentation:

- Create a complete state machine diagram
- Document all timing parameters
- Create a truth table for all traffic light combinations
- Provide a user manual explaining system operation

9 LAB SUBMISSION REQUIREMENTS

9.1 Deliverables

For each exercise, submit the following:

1. Design Documentation:

- State machine diagrams (in standard UML notation)
- Time sequence diagrams for critical operations

2. Implementation:

- Well-commented C source code
- Proteus simulation files
- Makefile or build script for compiling your code

3. Analysis:

- Data collection methodology
- Raw data from measurements

- Processed results with statistical analysis
 - Comparative analysis between different implementations
4. **Technical Report:**
- Executive summary
 - Detailed description of your approach
 - Justification for design decisions
 - Critical evaluation of results
 - Discussion of limitations and future improvements
 - References to all sources used

9.2 Grading Criteria

Your submission will be evaluated based on the following criteria:

- **Correctness (25%):** Does the system meet all specified requirements?
- **Design Quality (20%):** Is the software architecture well-structured, modular, and maintainable?
- **Implementation (20%):** Is the code efficient, well-commented, and free of obvious bugs?
- **Analysis (25%):** How thorough and insightful is your analysis of the system's performance?
- **Documentation (10%):** Is your submission well-documented and clearly presented?

9.3 Submission and Verification Guidelines

- The lab may be completed in teams of up to 5 students.
- All submissions will be in the form of a comprehensive technical report documenting your work.
- The lab report is due on **April 16, 2025**.
- In-person verification will be conducted during the lab session following the submission deadline.
- During verification, each team will demonstrate their working implementation in Proteus.
- **Important:** All team members will be individually assessed through verbal questioning to verify understanding of the concepts and implementation details.
- **Note:** If any team member cannot adequately explain the implementation or demonstrate understanding of the core concepts, the entire team will receive a failing grade for the assignment.
- This policy is strictly enforced to ensure all team members contribute meaningfully to the lab work.
- Late submissions will not be accepted except in extraordinary circumstances with prior approval.

Note: Start early and ask questions promptly if you encounter difficulties. The lab is designed to be challenging but achievable with proper time management and engagement with the material.